

ui.list 全面详细阐述

ui.list 是 NiceGUI 框架中基于 Quasar QList 组件的列表容器元素，核心用于承载 ui.item 系列子元素（包括 ui.item、ui.item_section、ui.item_label 等），支持灵活的结构组合、样式定制和交互绑定，适用于构建各类列表型界面（如普通文本列表、联系人列表、分类列表等）。以下从核心特性、使用示例、组件关联、属性与方法、继承关系等维度全面解析。

一、核心定位与基础特性

1. 核心作用

- 作为列表容器，统一管理多个 ui.item 子元素，提供一致的布局结构；
- 支持通过 props 快速配置列表整体样式（如边框、分隔线、紧凑模式等）；
- 兼容 NiceGUI 的事件绑定、动态更新、元素嵌套等核心能力。

2. 基础依赖

- 依赖 Quasar 组件库的 QList 底层实现，继承其基础样式与交互能力；
- 子元素通常为 ui.item 系列组件（ui.item 为列表项，ui.item_section 用于拆分列表项结构，ui.item_label 用于文本展示），需配合使用以实现复杂列表布局。

二、基础使用示例

ui.list 的使用需通过 with 语句嵌套子元素，核心分为“简单文本列表”和“复杂结构化列表”两类场景。

1. 简单文本列表（仅含 ui.item）

适用于仅展示纯文本的基础列表，通过 props 配置列表样式（如紧凑模式、分隔线）。

```
from nicegui import ui

# 创建带紧凑模式和分隔线的列表
with ui.list().props('dense separator'):
    ui.item('3 Apples')          # 列表项：纯文本内容
    ui.item('5 Bananas')
    ui.item('8 Strawberries')
    ui.item('13 walnuts')

ui.run()
```

效果：列表项垂直排列，带分隔线，文本紧凑显示，无额外复杂结构。

2. 复杂结构化列表（含分类、图标、多区域布局）

适用于需要分类标题、图标、多区域拆分的列表（如联系人列表），需结合 ui.item_label（分类标题）、ui.item_section（区域拆分）、ui.icon（图标）等组件。

```
from nicegui import ui
```

```
# 创建带边框和分隔线的列表
with ui.list().props('bordered separator'):
    # 列表分类标题（header 样式，加粗）
    ui.item_label('Contacts').props('header').classes('text-bold')
    ui.separator() # 分隔线（分割标题与列表项）

    # 列表项 1：含头像区、文本区、侧边图标区
    with ui.item(on_click=lambda: ui.notify('Selected contact 1')): # 点击事件
        # 左侧头像区（avatar props 固定尺寸）
        with ui.item_section().props('avatar'):
            ui.icon('person') # 图标作为头像
        # 中间文本区（主文本 + 辅助文本）
        with ui.item_section():
            ui.item_label('Nice Guy') # 主文本
            ui.item_label('name').props('caption') # 辅助文本（小标题样式）
        # 右侧侧边区（操作图标）
        with ui.item_section().props('side'):
            ui.icon('chat')

    # 列表项 2：结构与第一项一致
    with ui.item(on_click=lambda: ui.notify('Selected contact 2')):
        with ui.item_section().props('avatar'):
            ui.icon('person')
        with ui.item_section():
            ui.item_label('Nice Person')
            ui.item_label('name').props('caption')
        with ui.item_section().props('side'):
            ui.icon('chat')

ui.run()
```

效果：列表含“Contacts”分类标题，列表项分为左（头像）、中（文本）、右（操作图标）三区域，点击列表项弹出通知。

三、关键关联组件

ui.list 需依赖以下子组件实现复杂布局，核心关联关系如下：

组件名	作用	常用 props / 特性
ui.item	列表项容器，承载单个列表项的所有内容	<code>on_click</code> ：点击事件； <code>clickable</code> ：标记可点击
ui.item_section	拆分列表项区域（如左、中、右）	<code>avatar</code> ：头像区（固定尺寸）； <code>side</code> ：侧边区
ui.item_label	文本展示（主文本 / 辅助文本）	<code>header</code> ：标题样式； <code>caption</code> ：辅助文本样式
ui.separator	分隔线（分割分类标题与列表项）	-

核心逻辑: `ui.list` → 包含多个 `ui.item` → 每个 `ui.item` 包含多个 `ui.item_section` → 每个 `ui.item_section` 包含 `ui.item_label/ui.icon` 等内容。

四、核心属性 (Properties)

`ui.list` 的属性用于配置列表整体样式、状态和 DOM 相关信息，支持动态绑定和修改：

属性名	类型	说明
<code>classes</code>	<code>Classes[Self]</code>	自定义 HTML 类（支持 Tailwind/Quasar 类，用于样式定制）
<code>client</code>	<code>Client</code>	列表所属的客户端实例（框架内部使用，无需手动设置）
<code>html_id</code>	<code>str</code>	DOM 元素的 ID（版本 2.16.0 新增，用于直接操作 DOM）
<code>is_deleted</code>	<code>bool</code>	标记元素是否已删除（只读，框架内部维护）
<code>is_ignoring_events</code>	<code>bool</code>	标记元素是否忽略事件（只读，用于控制事件响应状态）
<code>parent_slot</code>	<code>Slot None</code>	父组件的插槽（可设置，用于插槽嵌套场景）
<code>props</code>	<code>Props[Self]</code>	Quasar 组件 props（核心样式配置，如 <code>bordered / dense / separator</code> ）
<code>style</code>	<code>Style[Self]</code>	自定义 CSS 样式（如 <code>width: 300px</code> ）
<code>visible</code>	<code>BindableProperty</code>	可见性（支持动态绑定，如绑定变量控制显示 / 隐藏）

常用 props 说明

props 名称	作用	示例
<code>bordered</code>	为列表添加边框	<code>ui.list().props('bordered')</code>
<code>dense</code>	紧凑模式（减少列表项间距）	<code>ui.list().props('dense')</code>
<code>separator</code>	显示列表项之间的分隔线	<code>ui.list().props('separator')</code>
<code>dark</code>	深色模式	<code>ui.list().props('dark')</code>

五、核心方法 (Methods)

`ui.list` 继承自 `Element` 类，提供丰富的方法用于动态操作、事件绑定、资源管理等，常用方法如下：

1. 元素操作相关

方法名	参数	作用
<code>clear()</code>	-	移除所有子元素（清空列表）
<code>delete()</code>	-	删除列表本身及所有子元素
<code>remove(element)</code>	<code>element: Element int</code>	移除指定子元素（传入元素实例或 ID）

方法名	参数	作用
move()	target_container/ target_index/ target_slot	移动列表到其他容器 / 指定位置

2. 样式与状态绑定

方法名	参数	作用
default_classes()	add/remove/toggle/replace	批量修改默认 HTML 类（如统一添加样式）
default_props()	add/remove	批量添加 / 移除 Quasar props（如默认添加 <code>bordered</code> ）
default_style()	add/remove/replace	批量修改默认 CSS 样式
set_visibility(visible)	visible: bool	手动设置列表可见性（True 显示 / False 隐藏）
bind_visibility()	target_object/ target_name/ forward/backward	双向绑定可见性（如绑定变量 <code>show_list</code> ）
bind_visibility_from()	target_object/ target_name	单向绑定可见性（从目标对象读取状态）
bind_visibility_to()	target_object/ target_name	单向绑定可见性（向目标对象同步状态）

3. 事件与资源管理

方法名	参数	作用
on(type, handler)	type: 事件名（如 'click'）； handler: 回调函数	绑定事件（如列表点击事件）
tooltip(text)	text: 提示文本	为列表添加鼠标悬浮提示
add_resource(path)	path: 资源路径	添加静态资源（如 CSS/JS 文件）
add_dynamic_resource(name, function)	动态资源（函数返回资源响应）	用于动态加载资源（如异步数据）
update()	-	强制更新客户端的列表显示（动态修改后调用）

4. 其他常用方法

方法名	参数	作用
ancestors(include_self)	是否包含自身	迭代获取所有父元素
descendants(include_self)	是否包含自身	迭代获取所有子元素
mark(*markers)	标记字符串（如 'contact-list'）	为列表添加标记（用于测试 / 元素查询）
get_computed_prop(prop_name)	属性名	异步获取计算属性（需 await）
run_method(name, *args)	方法名 / 参数	调用客户端方法（如 Quasar 组件的底层方法）

六、事件绑定示例

ui.list 支持绑定各类 DOM 事件（如点击、鼠标悬浮），也可通过子组件 ui.item 的 on_click 绑定列表项事件：

1. 列表整体点击事件

```
from nicegui import ui

with ui.list().on('click', lambda e: ui.notify('List clicked!')):
    ui.item('Item 1')
    ui.item('Item 2')

ui.run()
```

2. 单个列表项点击事件（推荐）

通过 ui.item 的 on_click 绑定，精准响应单个列表项的点击：

```
from nicegui import ui

with ui.list():
    ui.item('Select Me', on_click=lambda: ui.notify('Item selected!'))
    ui.item('Another Item', on_click=lambda: ui.notify('Another item clicked!'))

ui.run()
```

七、继承关系

ui.list 继承自 NiceGUI 的核心基类，具备基类的所有能力，继承链如下：

```
Element → Visibility → ui.list
```

- **Element**：所有 UI 元素的基类，提供 classes / props / style 等基础属性和 on() / update() 等核心方法；

- **Visibility**: 提供可见性控制相关方法（如 `bind_visibility()` / `set_visibility()`）。

八、高级用法场景

1. 动态添加 / 删除列表项

通过按钮触发列表项的动态增删，结合 `clear()` / `remove()` 方法：

```
from nicegui import ui

with ui.column():
    # 列表容器
    list_container = ui.list().props('bordered')

    # 动态添加列表项
    def add_item():
        with list_container:
            ui.item(f'New Item {len(list_container.children) + 1}')

    # 清空列表
    def clear_list():
        list_container.clear()

    ui.button('Add Item', on_click=add_item)
    ui.button('Clear List', on_click=clear_list)

ui.run()
```

2. 可见性动态绑定

通过变量绑定列表可见性，实现“显示 / 隐藏”切换：

```
from nicegui import ui

show_list = ui.bindable_value(True) # 可绑定变量

with ui.column():
    # 绑定可见性
    list_container = ui.list().props('bordered').bind_visibility(show_list)
    with list_container:
        ui.item('Item 1')
        ui.item('Item 2')

    # 切换可见性
    ui.switch('Show List', value=show_list).bind_value(show_list)

ui.run()
```

3. 自定义样式 (通过 classes/style)

使用 Tailwind 类或自定义 CSS 调整列表样式:

```
from nicegui import ui

# 自定义宽度、背景色、圆角的列表
with ui.list().classes('w-64 bg-gray-50 rounded-lg p-2')\
    .style('border: 2px solid #e5e7eb;'):
    ui.item('Styled Item 1').classes('text-blue-600')
    ui.item('Styled Item 2').classes('text-green-600')

ui.run()
```

九、注意事项

- props 兼容性:** `ui.list` 的 `props` 直接映射 Quasar QList 的 props, 使用时需参考 Quasar 文档 (如 `dense / bordered` 均为 QList 原生 props) ;
- 插槽使用:** 复杂场景下可通过 `add_slot()` 方法添加 Vue 插槽 (如自定义列表头部 / 尾部) , 但常规列表无需手动操作插槽;
- 事件冒泡:** 列表整体的 `click` 事件会与列表项的 `on_click` 事件冒泡, 如需区分, 可通过事件参数 `e.target` 判断触发源;
- 版本兼容性:** `html_id` 属性需版本 $\geq 2.16.0$, `bind_visibility` 的 `strict` 参数需版本 $\geq 3.0.0$, 使用时注意 NiceGUI 版本。

总结

`ui.list` 是 NiceGUI 中功能强大、灵活度高的列表容器组件, 核心优势在于:

- 结构清晰: 通过嵌套 `ui.item / ui.item_section` 实现复杂布局, 易于维护;
- 样式灵活: 支持 props/classes/style 三层样式定制, 适配各类 UI 需求;
- 交互丰富: 支持事件绑定、动态更新、可见性绑定等核心能力;
- 兼容性强: 继承框架基类能力, 可与其他 NiceGUI 组件 (如按钮、开关) 无缝配合。

适用于构建联系人列表、菜单列表、数据展示列表等各类场景, 是 NiceGUI 开发中高频使用的核心组件之一。